



COMP3153/9153

Algorithmic Verification

Lecture 10: (Bi-)Simulation, Abstraction and CEGAR

Model Equivalence

Let A and B be Kripke structures.

Question

When does $A \models \varphi \Leftrightarrow B \models \varphi$ for all LTL formulae φ ?

Model Equivalence

Let A and B be Kripke structures.

Question

When does $A \models \varphi \Leftrightarrow B \models \varphi$ for all LTL formulae φ ?

When A and B have the same behaviours. Why?

Model Equivalence

Let A and B be Kripke structures.

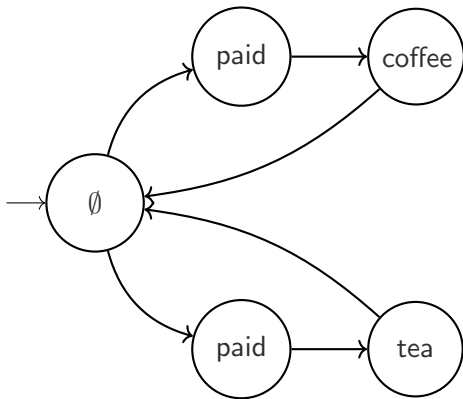
Question

When does $A \models \varphi \Leftrightarrow B \models \varphi$ for all LTL formulae φ ?

When A and B have the same behaviours. Why?

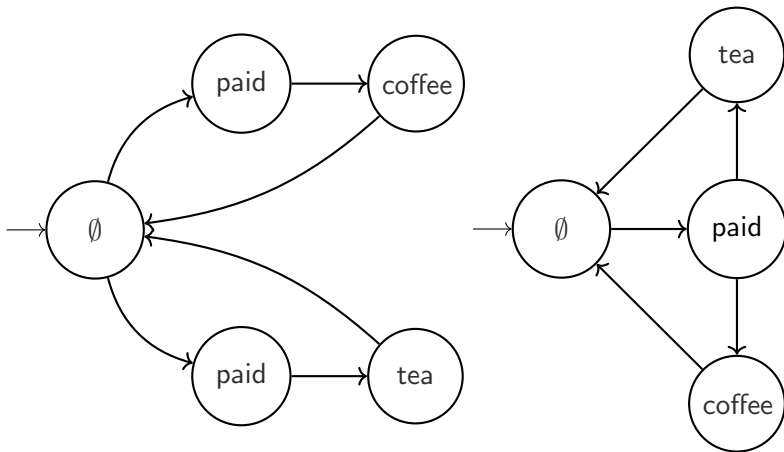
This is called *infinite completed trace equivalence*.

Limitations of Traces



Traces cannot distinguish these two models!

Limitations of Traces



Traces cannot distinguish these two models!

Model Equivalence

Question

When does $A \models \varphi \Leftrightarrow B \models \varphi$ for all **CTL** formulae φ ?

Model Equivalence

Question

When does $A \models \varphi \Leftrightarrow B \models \varphi$ for all CTL formulae φ ?

hmm...

Is it (only) when $A = B$ (*graph isomorphism*)?

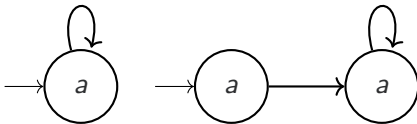
Model Equivalence

Question

When does $A \models \varphi \Leftrightarrow B \models \varphi$ for all **CTL** formulae φ ?

hmm...

Is it (only) when $A = B$ (*graph isomorphism*)?



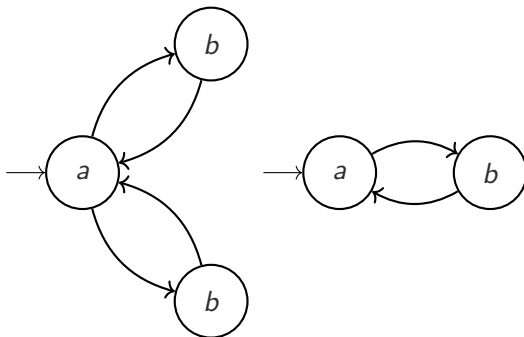
Nope!

Tree Equivalence?

Is it (only) when the two automata have the same computation tree?

Tree Equivalence?

Is it (only) when the two automata have the same computation tree?



Also no!

Bisimulations

Definition

A (strong) *bisimulation* between two automata A and B is defined as a relation $\mathcal{R} \subseteq Q_A \times Q_B$ which satisfies:

- If $s \mathcal{R} t$ then $L_A(s) = L_B(t)$

Bisimulations

Definition

A (strong) *bisimulation* between two automata A and B is defined as a relation $\mathcal{R} \subseteq Q_A \times Q_B$ which satisfies:

- If $s \mathcal{R} t$ then $L_A(s) = L_B(t)$
- If $s \mathcal{R} t$ and $s \xrightarrow{a} s'$ (with $a \in \Sigma_A, s' \in Q_A$) then there exists a $t' \in Q_B$ such that $t \xrightarrow{a} t'$ and $s' \mathcal{R} t'$.

Bisimulations

Definition

A (strong) *bisimulation* between two automata A and B is defined as a relation $\mathcal{R} \subseteq Q_A \times Q_B$ which satisfies:

- If $s \mathcal{R} t$ then $L_A(s) = L_B(t)$
- If $s \mathcal{R} t$ and $s \xrightarrow{a} s'$ (with $a \in \Sigma_A, s' \in Q_A$) then there exists a $t' \in Q_B$ such that $t \xrightarrow{a} t'$ and $s' \mathcal{R} t'$.
- If $s \mathcal{R} t$ and $t \xrightarrow{a} t'$ (with $a \in \Sigma_B, t' \in Q_B$) then there exists a $s' \in Q_A$ such that $s \xrightarrow{a} s'$ and $s' \mathcal{R} t'$.

Bisimulations

Definition

A (strong) *bisimulation* between two automata A and B is defined as a relation $\mathcal{R} \subseteq Q_A \times Q_B$ which satisfies:

- If $s \mathcal{R} t$ then $L_A(s) = L_B(t)$
- If $s \mathcal{R} t$ and $s \xrightarrow{a} s'$ (with $a \in \Sigma_A, s' \in Q_A$) then there exists a $t' \in Q_B$ such that $t \xrightarrow{a} t'$ and $s' \mathcal{R} t'$.
- If $s \mathcal{R} t$ and $t \xrightarrow{a} t'$ (with $a \in \Sigma_B, t' \in Q_B$) then there exists a $s' \in Q_A$ such that $s \xrightarrow{a} s'$ and $s' \mathcal{R} t'$.

Two automata are *bisimulation equivalent* or *bisimilar* iff there exists a bisimulation between their initial states.

Bisimulations

Definition

A (strong) *bisimulation* between two automata A and B is defined as a relation $\mathcal{R} \subseteq Q_A \times Q_B$ which satisfies:

- If $s \mathcal{R} t$ then $L_A(s) = L_B(t)$
- If $s \mathcal{R} t$ and $s \xrightarrow{a} s'$ (with $a \in \Sigma_A, s' \in Q_A$) then there exists a $t' \in Q_B$ such that $t \xrightarrow{a} t'$ and $s' \mathcal{R} t'$.
- If $s \mathcal{R} t$ and $t \xrightarrow{a} t'$ (with $a \in \Sigma_B, t' \in Q_B$) then there exists a $s' \in Q_A$ such that $s \xrightarrow{a} s'$ and $s' \mathcal{R} t'$.

Two automata are *bisimulation equivalent* or *bisimilar* iff there exists a bisimulation between their initial states.

Let's find bisimulations for the previous examples.

Bisimulations

Definition

A (strong) *bisimulation* between two automata A and B is defined as a relation $\mathcal{R} \subseteq Q_A \times Q_B$ which satisfies:

- If $s \mathcal{R} t$ then $L_A(s) = L_B(t)$
- If $s \mathcal{R} t$ and $s \xrightarrow{a} s'$ (with $a \in \Sigma_A, s' \in Q_A$) then there exists a $t' \in Q_B$ such that $t \xrightarrow{a} t'$ and $s' \mathcal{R} t'$.
- If $s \mathcal{R} t$ and $t \xrightarrow{a} t'$ (with $a \in \Sigma_B, t' \in Q_B$) then there exists a $s' \in Q_A$ such that $s \xrightarrow{a} s'$ and $s' \mathcal{R} t'$.

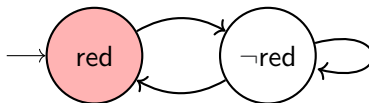
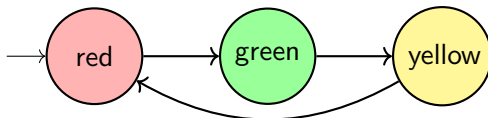
Two automata are *bisimulation equivalent* or *bisimilar* iff there exists a bisimulation between their initial states.

Let's find bisimulations for the previous examples.

Result

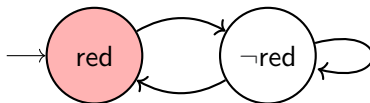
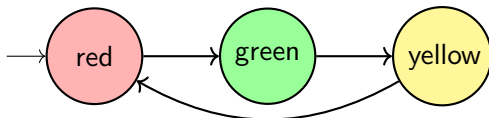
For two finitely-branching automata A and B , $A \models \varphi \Leftrightarrow B \models \varphi$ for all CTL formulae φ iff they are *bisimilar*.

Simulation



Are these bisimilar?

Simulation



Are these bisimilar?

No, but one **simulates** the other.

Simulation Relations

Definition

A *simulation* of an automaton C by an automaton A is defined as a relation $\mathcal{S} \subseteq Q_C \times Q_A$ which satisfies:

- If $s \mathcal{S} t$ then $L_C(s) \cap L_A = L_A(t)$

Simulation Relations

Definition

A *simulation* of an automaton C by an automaton A is defined as a relation $\mathcal{S} \subseteq Q_C \times Q_A$ which satisfies:

- If $s \mathcal{S} t$ then $L_C(s) \cap L_A = L_A(t)$
- If $s \mathcal{S} t$ and $s \xrightarrow{a} s'$ (with $a \in \Sigma_C, s' \in Q_C$) then there exists a $t' \in Q_A$ such that $t \xrightarrow{a} t'$ and $s' \mathcal{R} t'$.

Simulation Relations

Definition

A *simulation* of an automaton C by an automaton A is defined as a relation $\mathcal{S} \subseteq Q_C \times Q_A$ which satisfies:

- If $s \mathcal{S} t$ then $L_C(s) \cap L_A = L_A(t)$
- If $s \mathcal{S} t$ and $s \xrightarrow{a} s'$ (with $a \in \Sigma_C, s' \in Q_C$) then there exists a $t' \in Q_A$ such that $t \xrightarrow{a} t'$ and $s' \mathcal{R} t'$.

The automaton A is an *abstraction* of the concrete automaton C iff A simulates C . This is sometimes written $A \sqsubseteq C$.

Simulation Relations

Definition

A *simulation* of an automaton C by an automaton A is defined as a relation $\mathcal{S} \subseteq Q_C \times Q_A$ which satisfies:

- If $s \mathcal{S} t$ then $L_C(s) \cap L_A = L_A(t)$
- If $s \mathcal{S} t$ and $s \xrightarrow{a} s'$ (with $a \in \Sigma_C, s' \in Q_C$) then there exists a $t' \in Q_A$ such that $t \xrightarrow{a} t'$ and $s' \mathcal{R} t'$.

The automaton A is an *abstraction* of the concrete automaton C iff A simulates C . This is sometimes written $A \sqsubseteq C$.

Abstraction and Traces

If $A \sqsubseteq C$, then every trace of C *restricted* to L_A is a trace of A .

$$\sigma_1\sigma_2\sigma_3 \cdots \in \text{Traces}(C)$$

$$\Rightarrow$$

$$(\sigma_1 \cap L_A)(\sigma_2 \cap L_A)(\sigma_3 \cap L_A) \cdots \in \text{Traces}(A)$$

Essential Property of Simulations

Let $\mathcal{A}$ be a simulation relation, showing that $X \sqsubseteq Y$.

Then for every run $\rho_1\rho_2\rho_3\cdots \in Y$ is a run of X by applying the simulation relation as an *abstraction mapping*:

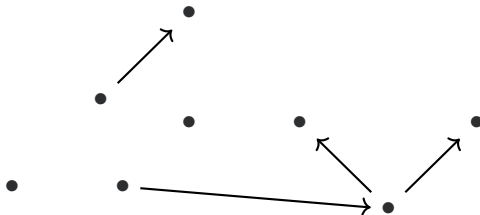
$$\mathcal{A}(\rho_1)\mathcal{A}(\rho_2)\mathcal{A}(\rho_3)\cdots \in X$$

Essential Property of Simulations

Let $\mathcal{A}$ be a simulation relation, showing that $X \sqsubseteq Y$.

Then for every run $\rho_1\rho_2\rho_3\cdots \in Y$ is a run of X by applying the simulation relation as an *abstraction mapping*:

$$\mathcal{A}(\rho_1)\mathcal{A}(\rho_2)\mathcal{A}(\rho_3)\cdots \in X$$

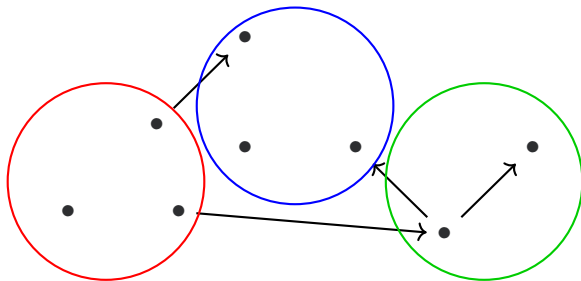


Essential Property of Simulations

Let $\mathcal{A}$ be a simulation relation, showing that $X \sqsubseteq Y$.

Then for every run $\rho_1\rho_2\rho_3\cdots \in Y$ is a run of X by applying the simulation relation as an *abstraction mapping*:

$$\mathcal{A}(\rho_1)\mathcal{A}(\rho_2)\mathcal{A}(\rho_3)\cdots \in X$$

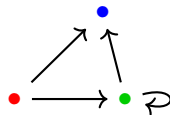
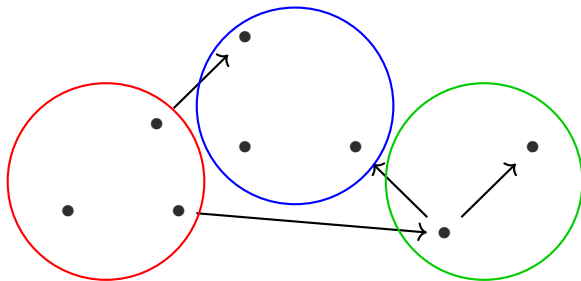


Essential Property of Simulations

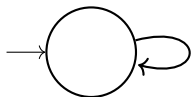
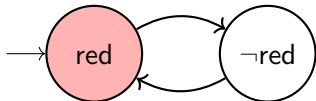
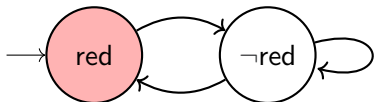
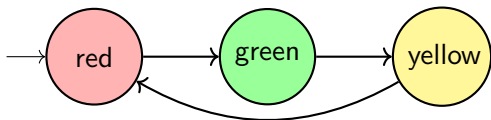
Let $\mathcal{A}$ be a simulation relation, showing that $X \sqsubseteq Y$.

Then for every run $\rho_1\rho_2\rho_3\cdots \in Y$ is a run of X by applying the simulation relation as an *abstraction mapping*:

$$\mathcal{A}(\rho_1)\mathcal{A}(\rho_2)\mathcal{A}(\rho_3)\cdots \in X$$



Comparing Automata



What are the simulations between these?

Reducing State Space

We want abstraction to **shrink the state space** for model checking. To do this, we need a guarantee that any property we prove about an abstraction applies just as well to the concrete model.

Reducing State Space

We want abstraction to **shrink the state space** for model checking. To do this, we need a guarantee that any property we prove about an abstraction applies just as well to the concrete model.

Universal Properties

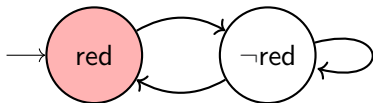
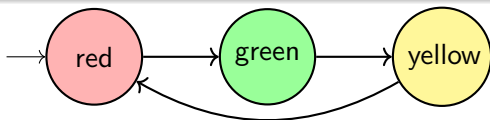
Given $A \sqsubseteq C$, which φ satisfy $A \models \varphi \Rightarrow C \models \varphi$?

Reducing State Space

We want abstraction to **shrink the state space** for model checking. To do this, we need a guarantee that any property we prove about an abstraction applies just as well to the concrete model.

Universal Properties

Given $A \sqsubseteq C$, which φ satisfy $A \models \varphi \Rightarrow C \models \varphi$?



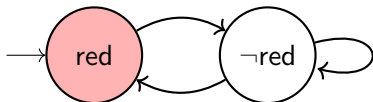
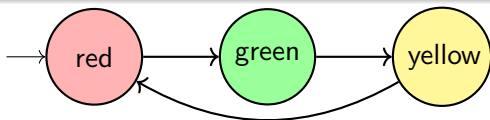
AG AF $\neg \text{red}$?

Reducing State Space

We want abstraction to **shrink the state space** for model checking. To do this, we need a guarantee that any property we prove about an abstraction applies just as well to the concrete model.

Universal Properties

Given $A \sqsubseteq C$, which φ satisfy $A \models \varphi \Rightarrow C \models \varphi$?



AG AF ¬red? **Works!**

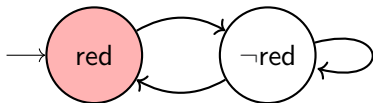
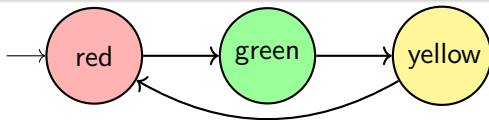
¬AG AF red?

Reducing State Space

We want abstraction to **shrink the state space** for model checking. To do this, we need a guarantee that any property we prove about an abstraction applies just as well to the concrete model.

Universal Properties

Given $A \sqsubseteq C$, which φ satisfy $A \models \varphi \Rightarrow C \models \varphi$?



AG AF ¬red? **Works!**

¬AG AF red? **Doesn't work!**

Universal CTL

Negation Normal Form

φ is in negation normal form (NNF), written $\hat{\varphi}$, if all negations are applied only to atomic props. All formulae have a NNF equivalent.

Universal CTL

Negation Normal Form

φ is in negation normal form (NNF), written $\hat{\varphi}$, if all negations are applied only to atomic props. All formulae have a NNF equivalent.

ACTL

φ is a formula in ACTL, the *Universal CTL*, iff its *negation normal form*, $\hat{\varphi}$, does not contain **E**.

Universal CTL

Negation Normal Form

φ is in negation normal form (NNF), written $\hat{\varphi}$, if all negations are applied only to atomic props. All formulae have a NNF equivalent.

ACTL

φ is a formula in ACTL, the *Universal CTL*, iff its *negation normal form*, $\hat{\varphi}$, does not contain **E**.

Example

- AGp

Universal CTL

Negation Normal Form

φ is in negation normal form (NNF), written $\hat{\varphi}$, if all negations are applied only to atomic props. All formulae have a NNF equivalent.

ACTL

φ is a formula in ACTL, the *Universal CTL*, iff its *negation normal form*, $\hat{\varphi}$, does not contain **E**.

Example

- $\text{AG}p$
- $\text{AG AF}p$

Universal CTL

Negation Normal Form

φ is in negation normal form (NNF), written $\hat{\varphi}$, if all negations are applied only to atomic props. All formulae have a NNF equivalent.

ACTL

φ is a formula in ACTL, the *Universal CTL*, iff its *negation normal form*, $\hat{\varphi}$, does not contain **E**.

Example

- **AG** p
- **AG AF** p
- **EF** p

Universal CTL

Negation Normal Form

φ is in negation normal form (NNF), written $\hat{\varphi}$, if all negations are applied only to atomic props. All formulae have a NNF equivalent.

ACTL

φ is a formula in ACTL, the *Universal CTL*, iff its *negation normal form*, $\hat{\varphi}$, does not contain **E**.

Example

- **AG** p
- **AG AF** p
- **EF** p — Nope!

Negation Normal Form

$$\neg \mathbf{AF} \varphi \equiv \mathbf{EG} \neg \varphi$$

$$\neg \mathbf{EF} \varphi \equiv \mathbf{AG} \neg \varphi$$

$$\neg \mathbf{AG} \varphi \equiv \mathbf{EF} \neg \varphi$$

$$\neg \mathbf{EG} \varphi \equiv \mathbf{AF} \neg \varphi$$

$$\neg \mathbf{AX} \varphi \equiv \mathbf{EX} \neg \varphi$$

$$\neg \mathbf{EX} \varphi \equiv \mathbf{AX} \neg \varphi$$

$$\neg \mathbf{E}(\varphi \mathbf{U} \psi) \equiv$$

Negation Normal Form

$$\neg \mathbf{AF} \varphi \equiv \mathbf{EG} \neg \varphi$$

$$\neg \mathbf{EF} \varphi \equiv \mathbf{AG} \neg \varphi$$

$$\neg \mathbf{AG} \varphi \equiv \mathbf{EF} \neg \varphi$$

$$\neg \mathbf{EG} \varphi \equiv \mathbf{AF} \neg \varphi$$

$$\neg \mathbf{AX} \varphi \equiv \mathbf{EX} \neg \varphi$$

$$\neg \mathbf{EX} \varphi \equiv \mathbf{AX} \neg \varphi$$

$$\neg \mathbf{E}(\varphi \mathbf{U} \psi) \equiv \mathbf{A}(\neg \varphi \mathbf{R} \neg \psi)$$

$$\neg \mathbf{A}(\varphi \mathbf{U} \psi) \equiv \mathbf{E}(\neg \varphi \mathbf{R} \neg \psi)$$

Negation Normal Form

$$\neg \mathbf{AF} \varphi \equiv \mathbf{EG} \neg \varphi$$

$$\neg \mathbf{EF} \varphi \equiv \mathbf{AG} \neg \varphi$$

$$\neg \mathbf{AG} \varphi \equiv \mathbf{EF} \neg \varphi$$

$$\neg \mathbf{EG} \varphi \equiv \mathbf{AF} \neg \varphi$$

$$\neg \mathbf{AX} \varphi \equiv \mathbf{EX} \neg \varphi$$

$$\neg \mathbf{EX} \varphi \equiv \mathbf{AX} \neg \varphi$$

$$\neg \mathbf{E}(\varphi \mathbf{U} \psi) \equiv \mathbf{A}(\neg \varphi \mathbf{R} \neg \psi)$$

$$\neg \mathbf{A}(\varphi \mathbf{U} \psi) \equiv \mathbf{E}(\neg \varphi \mathbf{R} \neg \psi)$$

Release Operator

The temporal operator $\varphi \mathbf{R} \psi$ says that ψ will not become false unless φ happens first.

$$\sigma \models \varphi \mathbf{R} \psi \iff \forall n \geq 0. (\forall 0 \leq k < n. \sigma|_k \not\models \varphi) \Rightarrow \sigma|_n \models \psi$$

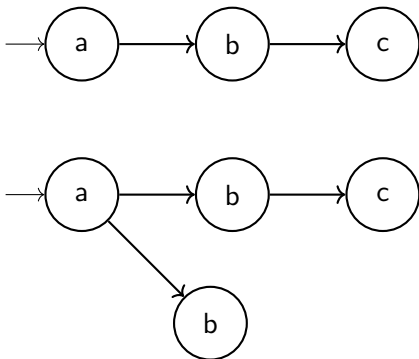
A and **E** variants in CTL follow the usual pattern.

Bisimulation and simulation

Suppose that $A \sqsubseteq B$ and $B \sqsubseteq A$. Does that mean A is bisimilar to B ?

Bisimulation and simulation

Suppose that $A \sqsubseteq B$ and $B \sqsubseteq A$. Does that mean A is bisimilar to B ?



Nope! This is another equivalence called *simulation equivalence*. Because of the abstraction result, ACTL is the logic that characterises simulation equivalence.

The Linear-time Branching-time Spectrum

Coarseness of Equivalences

- Graph isomorphism is finer (distinguishes more models) in than bisimilarity.

The Linear-time Branching-time Spectrum

Coarseness of Equivalences

- Graph isomorphism is finer (distinguishes more models) in than bisimilarity.
- Bisimilarity is finer than simulation equivalence.

The Linear-time Branching-time Spectrum

Coarseness of Equivalences

- Graph isomorphism is finer (distinguishes more models) in than bisimilarity.
- Bisimilarity is finer than simulation equivalence.
- Bisimilarity is finer than completed infinite trace equivalence.

The Linear-time Branching-time Spectrum

Coarseness of Equivalences

- Graph isomorphism is finer (distinguishes more models) in than bisimilarity.
- Bisimilarity is finer than simulation equivalence.
- Bisimilarity is finer than completed infinite trace equivalence.
- Partial trace equivalence (sets of finite-length traces) is coarser than all of the above.

The Linear-time Branching-time Spectrum

Coarseness of Equivalences

- Graph isomorphism is finer (distinguishes more models) in than bisimilarity.
- Bisimilarity is finer than simulation equivalence.
- Bisimilarity is finer than completed infinite trace equivalence.
- Partial trace equivalence (sets of finite-length traces) is coarser than all of the above.

There are **many, many** more equivalences.

The Linear-time Branching-time Spectrum

Coarseness of Equivalences

- Graph isomorphism is finer (distinguishes more models) in than bisimilarity.
- Bisimilarity is finer than simulation equivalence.
- Bisimilarity is finer than completed infinite trace equivalence.
- Partial trace equivalence (sets of finite-length traces) is coarser than all of the above.

There are **many, many** more equivalences.

Rob van Glabbeek categorised all of these equivalences and more into the *linear-time branching-time spectrum*, which is a major focus of his (former) course at this university, **COMP6752**.

Model Checking with Abstractions

Abstractions typically have a smaller state space, so it is advantageous to try to model check with abstractions rather than a concrete model.

Model Checking with Abstractions

Abstractions typically have a smaller state space, so it is advantageous to try to model check with abstractions rather than a concrete model.

We need:

- To know that properties that hold for our abstractions hold for our model — true for all $\varphi \in \text{ACTL}$.

Model Checking with Abstractions

Abstractions typically have a smaller state space, so it is advantageous to try to model check with abstractions rather than a concrete model.

We need:

- To know that properties that hold for our abstractions hold for our model — true for all $\varphi \in \text{ACTL}$.
- To know that when our properties **don't** hold for our abstractions, they don't hold for our model — not true in general!

Model Checking with Abstractions

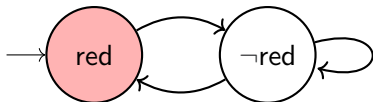
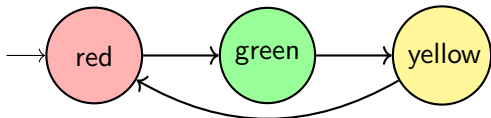
Abstractions typically have a smaller state space, so it is advantageous to try to model check with abstractions rather than a concrete model.

We need:

- To know that properties that hold for our abstractions hold for our model — true for all $\varphi \in \text{ACTL}$.
- To know that when our properties **don't** hold for our abstractions, they don't hold for our model — not true in general!

We need to pick the abstraction **based on** the properties we care about, and if necessary change our abstraction on the fly based on the results we see.

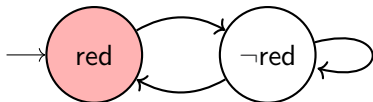
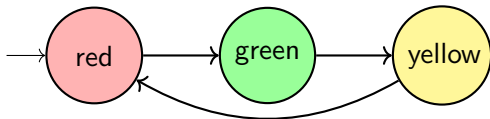
Model Checking with Abstractions



Consider the following ACTL formulae:

- **AG** ($\text{red} \Rightarrow \mathbf{AX} \neg\text{red}$)

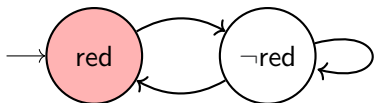
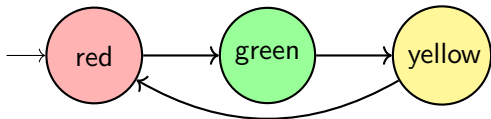
Model Checking with Abstractions



Consider the following ACTL formulae:

- **AG** ($\text{red} \Rightarrow \mathbf{AX} \neg\text{red}$)
- **AG** ($\text{red} \Rightarrow \mathbf{AX AX} \text{red}$)

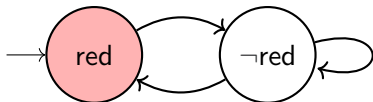
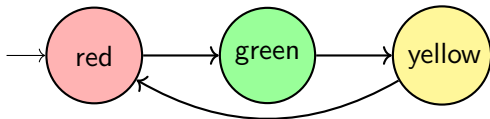
Model Checking with Abstractions



Consider the following ACTL formulae:

- **AG** ($\text{red} \Rightarrow \mathbf{AX} \neg\text{red}$)
- **AG** ($\text{red} \Rightarrow \mathbf{AX AX} \text{red}$)
- **AG** ($\text{red} \Rightarrow \mathbf{AX AX AX} \text{red}$)

Model Checking with Abstractions



Consider the following ACTL formulae:

- **AG** ($\text{red} \Rightarrow \mathbf{AX} \neg\text{red}$)
- **AG** ($\text{red} \Rightarrow \mathbf{AX AX} \text{red}$)
- **AG** ($\text{red} \Rightarrow \mathbf{AX AX AX} \text{red}$)

We know that if $A \sqsubseteq C$ then $(A \models \varphi) \Rightarrow (C \models \varphi)$ for $\varphi \in \text{ACTL}$,
but what about if $A \not\models \varphi$?

Counterexamples

Note

If $A \not\models \varphi$ for some $\varphi \in \text{ACTL}$, then there exists a run that serves as a *counterexample* to the formula φ .

- If $A \not\models \varphi$, that tells us *either* that $C \not\models \varphi$ or that our abstraction is not precise enough — the counterexample will be *spurious*.

Counterexamples

Note

If $A \not\models \varphi$ for some $\varphi \in \text{ACTL}$, then there exists a run that serves as a *counterexample* to the formula φ .

- If $A \not\models \varphi$, that tells us *either* that $C \not\models \varphi$ or that our abstraction is not precise enough — the counterexample will be *spurious*.
- **Our approach:** To check if our counterexample is spurious, convert it to a concrete run $\in C$.

Abstract to Concrete Run

Let α be our abstraction mapping $Q_C \rightarrow Q_A$ and our run be $q_0 q_1 q_2 \dots$. We apply the mapping in **reverse**, α^{-1} , and try to find a concrete run starting from our initial state I_C according to transition relation δ_C :

$$\begin{aligned} S_0 &= I_C \cap \alpha^{-1}(q_0) \\ S_1 &= \delta_C(S_0) \cap \alpha^{-1}(q_1) \\ S_2 &= \delta_C(S_1) \cap \alpha^{-1}(q_2) \quad \text{etc..} \end{aligned}$$

Abstract to Concrete Run

Let α be our abstraction mapping $Q_C \rightarrow Q_A$ and our run be $q_0 q_1 q_2 \dots$. We apply the mapping in **reverse**, α^{-1} , and try to find a concrete run starting from our initial state I_C according to transition relation δ_C :

$$S_0 = I_C \cap \alpha^{-1}(q_0)$$

$$S_1 = \delta_C(S_0) \cap \alpha^{-1}(q_1)$$

$$S_2 = \delta_C(S_1) \cap \alpha^{-1}(q_2) \quad \text{etc..}$$

If there is such a run (i.e. no $S_i = \emptyset$), the run is **not spurious**.

Abstract to Concrete Run

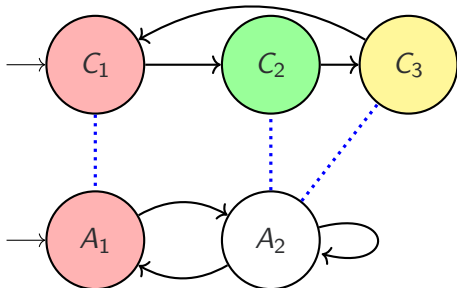
Let α be our abstraction mapping $Q_C \rightarrow Q_A$ and our run be $q_0 q_1 q_2 \dots$. We apply the mapping in **reverse**, α^{-1} , and try to find a concrete run starting from our initial state I_C according to transition relation δ_C :

$$S_0 = I_C \cap \alpha^{-1}(q_0)$$

$$S_1 = \delta_C(S_0) \cap \alpha^{-1}(q_1)$$

$$S_2 = \delta_C(S_1) \cap \alpha^{-1}(q_2) \quad \text{etc..}$$

If there is such a run (i.e. no $S_i = \emptyset$), the run is **not spurious**.



Example

AG (red $\Rightarrow$ **AX AX** red)

Abstract to Concrete Run

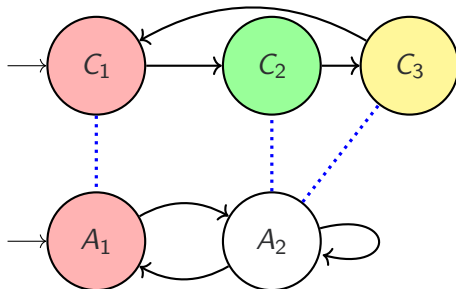
Let α be our abstraction mapping $Q_C \rightarrow Q_A$ and our run be $q_0 q_1 q_2 \dots$. We apply the mapping in **reverse**, α^{-1} , and try to find a concrete run starting from our initial state I_C according to transition relation δ_C :

$$S_0 = I_C \cap \alpha^{-1}(q_0)$$

$$S_1 = \delta_C(S_0) \cap \alpha^{-1}(q_1)$$

$$S_2 = \delta_C(S_1) \cap \alpha^{-1}(q_2) \quad \text{etc..}$$

If there is such a run (i.e. no $S_i = \emptyset$), the run is **not spurious**.



Example

AG (red $\Rightarrow$ **AX AX** red)

Counterexample: $A_1 A_2 A_2$

Abstract to Concrete Run

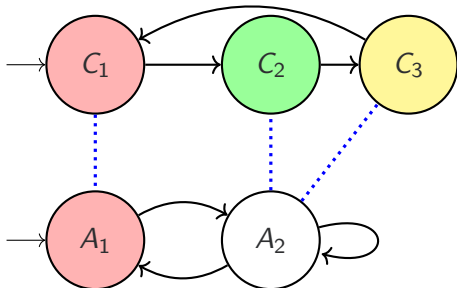
Let α be our abstraction mapping $Q_C \rightarrow Q_A$ and our run be $q_0 q_1 q_2 \dots$. We apply the mapping in **reverse**, α^{-1} , and try to find a concrete run starting from our initial state I_C according to transition relation δ_C :

$$S_0 = I_C \cap \alpha^{-1}(q_0)$$

$$S_1 = \delta_C(S_0) \cap \alpha^{-1}(q_1)$$

$$S_2 = \delta_C(S_1) \cap \alpha^{-1}(q_2) \quad \text{etc..}$$

If there is such a run (i.e. no $S_i = \emptyset$), the run is **not spurious**.



Example

AG (red $\Rightarrow$ **AX AX** red)

Counterexample: $A_1 A_2 A_2$

$$\alpha^{-1}(A_1 A_2 A_2)$$

$$= \{C_1\}\{C_2, C_3\}\{C_2, C_3\}$$

Abstract to Concrete Run

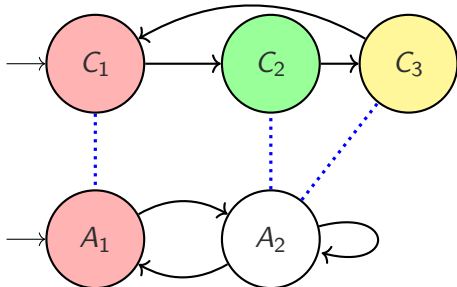
Let α be our abstraction mapping $Q_C \rightarrow Q_A$ and our run be $q_0 q_1 q_2 \dots$. We apply the mapping in **reverse**, α^{-1} , and try to find a concrete run starting from our initial state I_C according to transition relation δ_C :

$$S_0 = I_C \cap \alpha^{-1}(q_0)$$

$$S_1 = \delta_C(S_0) \cap \alpha^{-1}(q_1)$$

$$S_2 = \delta_C(S_1) \cap \alpha^{-1}(q_2) \quad \text{etc..}$$

If there is such a run (i.e. no $S_i = \emptyset$), the run is **not spurious**.



Example

AG (red $\Rightarrow$ **AX AX** red)

Counterexample: $A_1 A_2 A_2$
 $\alpha^{-1}(A_1 A_2 A_2)$

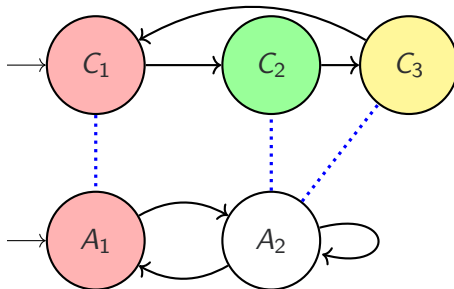
$$= \{C_1\} \{C_2, C_3\} \{C_2, C_3\}$$

There is a run

$$C_1 \xrightarrow{\delta_C} C_2 \xrightarrow{\delta_C} C_3$$

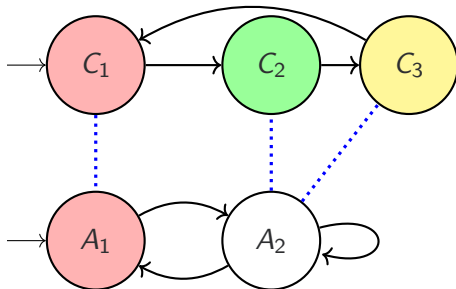
$\therefore$ **Not spurious.**

Spurious Counterexamples



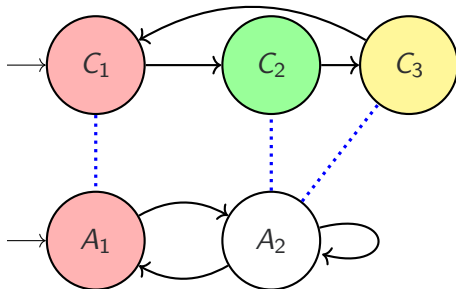
AG (red $\Rightarrow$ **AX AX AX** red)

Spurious Counterexamples



AG (red $\Rightarrow$ **AX AX AX** red) **Counterexample:** $A_1 A_2 A_2 A_2$

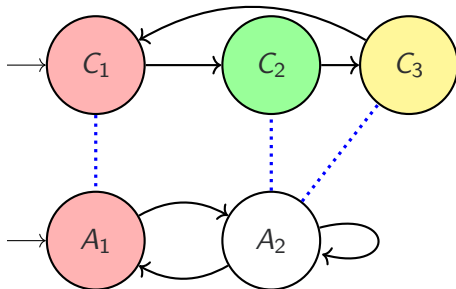
Spurious Counterexamples



AG (red $\Rightarrow$ **AX AX AX** red) **Counterexample:** $A_1 A_2 A_2 A_2$

$$S_0 = I_C \cap \alpha^{-1}(A_1) =$$

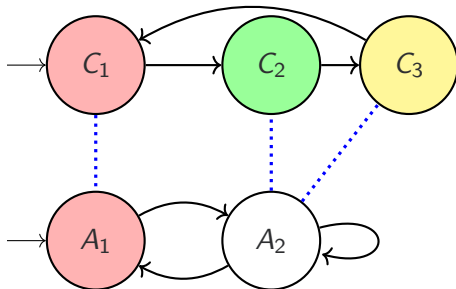
Spurious Counterexamples



AG (red $\Rightarrow$ **AX AX AX** red) **Counterexample:** $A_1 A_2 A_2 A_2$

$$S_0 = I_C \cap \alpha^{-1}(A_1) = \{C_1\} \cap \{C_1\} =$$

Spurious Counterexamples

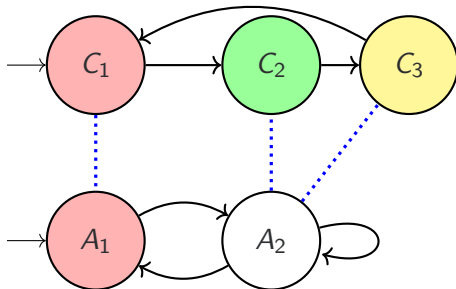


AG (red $\Rightarrow$ **AX AX AX** red) **Counterexample:** $A_1 A_2 A_2 A_2$

$$S_0 = I_C \cap \alpha^{-1}(A_1) = \{C_1\} \cap \{C_1\} = \{C_1\}$$

$$S_1 = \delta_C(S_0) \cap \alpha^{-1}(A_2)$$

Spurious Counterexamples



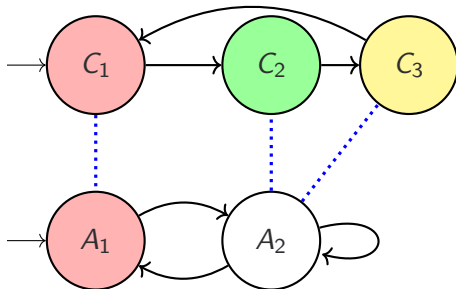
AG (red $\Rightarrow$ **AX AX AX** red) **Counterexample:** $A_1 A_2 A_2 A_2$

$$S_0 = I_C \cap \alpha^{-1}(A_1) = \{C_1\} \cap \{C_1\} = \{C_1\}$$

$$S_1 = \delta_C(S_0) \cap \alpha^{-1}(A_2) = \{C_2\} \cap \{C_2, C_3\} = \{C_2\}$$

$$S_2 = \delta_C(S_1) \cap \alpha^{-1}(A_2)$$

Spurious Counterexamples



AG (red $\Rightarrow$ **AX AX AX** red) **Counterexample:** $A_1 A_2 A_2 A_2$

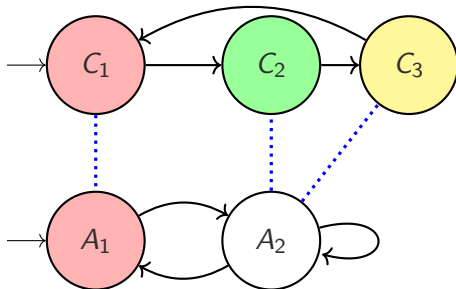
$$S_0 = I_C \cap \alpha^{-1}(A_1) = \{C_1\} \cap \{C_1\} = \{C_1\}$$

$$S_1 = \delta_C(S_0) \cap \alpha^{-1}(A_2) = \{C_2\} \cap \{C_2, C_3\} = \{C_2\}$$

$$S_2 = \delta_C(S_1) \cap \alpha^{-1}(A_2) = \{C_3\} \cap \{C_2, C_3\} = \{C_3\}$$

$$S_3 = \delta_C(S_2) \cap \alpha^{-1}(A_2)$$

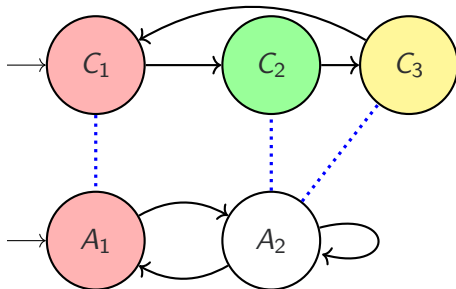
Spurious Counterexamples



AG (red $\Rightarrow$ **AX AX AX** red) **Counterexample:** $A_1 A_2 A_2 A_2$

$$\begin{aligned}
 S_0 &= I_C \cap \alpha^{-1}(A_1) &= \{C_1\} \cap \{C_1\} &= \{C_1\} \\
 S_1 &= \delta_C(S_0) \cap \alpha^{-1}(A_2) &= \{C_2\} \cap \{C_2, C_3\} &= \{C_2\} \\
 S_2 &= \delta_C(S_1) \cap \alpha^{-1}(A_2) &= \{C_3\} \cap \{C_2, C_3\} &= \{C_3\} \\
 S_3 &= \delta_C(S_2) \cap \alpha^{-1}(A_2) &= \{C_1\} \cap \{C_2, C_3\} &=
 \end{aligned}$$

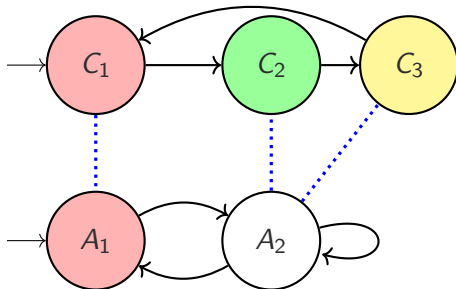
Spurious Counterexamples



AG (red $\Rightarrow$ **AX AX AX** red) **Counterexample:** $A_1 A_2 A_2 A_2$

$$\begin{aligned}
 S_0 &= I_C \cap \alpha^{-1}(A_1) &= \{C_1\} \cap \{C_1\} &= \{C_1\} \\
 S_1 &= \delta_C(S_0) \cap \alpha^{-1}(A_2) &= \{C_2\} \cap \{C_2, C_3\} &= \{C_2\} \\
 S_2 &= \delta_C(S_1) \cap \alpha^{-1}(A_2) &= \{C_3\} \cap \{C_2, C_3\} &= \{C_3\} \\
 S_3 &= \delta_C(S_2) \cap \alpha^{-1}(A_2) &= \{C_1\} \cap \{C_2, C_3\} &= \emptyset
 \end{aligned}$$

Spurious Counterexamples



AG (red $\Rightarrow$ **AX AX AX** red) **Counterexample:** $A_1 A_2 A_2 A_2$

$$\begin{aligned}
 S_0 &= I_C \cap \alpha^{-1}(A_1) &= \{C_1\} \cap \{C_1\} &= \{C_1\} \\
 S_1 &= \delta_C(S_0) \cap \alpha^{-1}(A_2) &= \{C_2\} \cap \{C_2, C_3\} &= \{C_2\} \\
 S_2 &= \delta_C(S_1) \cap \alpha^{-1}(A_2) &= \{C_3\} \cap \{C_2, C_3\} &= \{C_3\} \\
 S_3 &= \delta_C(S_2) \cap \alpha^{-1}(A_2) &= \{C_1\} \cap \{C_2, C_3\} &= \emptyset
 \end{aligned}$$

There is no concrete run — this counterexample is spurious. Our abstraction is too **imprecise**.

Abstraction Refinement

Definition

An abstraction mapping α generates an equivalence relation on states $\equiv_\alpha$ where $q \equiv_\alpha q' \Leftrightarrow \alpha(q) = \alpha(q')$.

Abstraction Refinement

Definition

An abstraction mapping α generates an equivalence relation on states $\equiv_\alpha$ where $q \equiv_\alpha q' \Leftrightarrow \alpha(q) = \alpha(q')$.

Consider two abstractions $\alpha : Q_C \rightarrow Q_A$ and $\alpha' : Q_C \rightarrow Q_B$.

We say that α' **refines** α iff $\equiv_{\alpha'} \subseteq \equiv_\alpha$.

Similarly, we say α' **strictly refines** α iff $\equiv_{\alpha'} \subsetneq \equiv_\alpha$

Informal Notion

We previously considered abstractions as grouping together concrete states into equivalence classes. We can refine abstractions by **splitting** those equivalence classes.

Abstraction Refinement

We have a spurious counterexample $q_1q_2q_3\dots$

Which classes should we split up in our new abstraction?

Abstraction Refinement

We have a spurious counterexample $q_1q_2q_3\dots$

Which classes should we split up in our new abstraction?

Counterexample Guidance

For each q_i in our counterexample, the class of concrete states it is abstracting is $\alpha^{-1}(q_i)$.

Abstraction Refinement

We have a spurious counterexample $q_1 q_2 q_3 \dots$

Which classes should we split up in our new abstraction?

Counterexample Guidance

For each q_i in our counterexample, the class of concrete states it is abstracting is $\alpha^{-1}(q_i)$.

We will split this class into two sets:

- 1 Those that follow directly from the previous state:

$$\alpha^{-1}(q_i) \cap \delta_C(S_{i-1})$$

Abstraction Refinement

We have a spurious counterexample $q_1 q_2 q_3 \dots$

Which classes should we split up in our new abstraction?

Counterexample Guidance

For each q_i in our counterexample, the class of concrete states it is abstracting is $\alpha^{-1}(q_i)$.

We will split this class into two sets:

- 1 Those that follow directly from the previous state:
 $\alpha^{-1}(q_i) \cap \delta_C(S_{i-1})$
- 2 Those that don't: $\alpha^{-1}(q_i) \setminus \delta_C(S_{i-1})$

Abstraction Refinement

We have a spurious counterexample $q_1 q_2 q_3 \dots$

Which classes should we split up in our new abstraction?

Counterexample Guidance

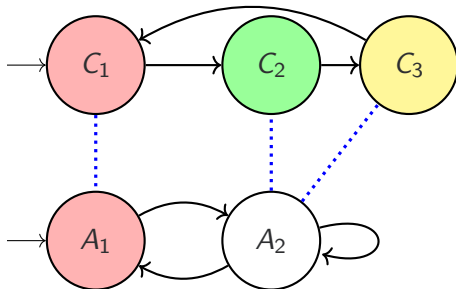
For each q_i in our counterexample, the class of concrete states it is abstracting is $\alpha^{-1}(q_i)$.

We will split this class into two sets:

- 1 Those that follow directly from the previous state:
 $\alpha^{-1}(q_i) \cap \delta_C(S_{i-1})$
- 2 Those that don't: $\alpha^{-1}(q_i) \setminus \delta_C(S_{i-1})$

The resulting classes will form the new, refined abstraction of our model. If both of these sets are non-empty, we split the state q_i into two states, one for each set.

Example



AG (red $\Rightarrow$ **AX AX AX** red) **Counterexample:** $A_1 A_2 A_2 A_2$

$$S_0 = I_C \cap \alpha^{-1}(A_1) = \{C_1\} \cap \{C_1\} = \{C_1\}$$

$$S_1 = \delta_C(S_0) \cap \alpha^{-1}(A_2) = \{C_2\} \cap \{C_2, C_3\} = \{C_2\}$$

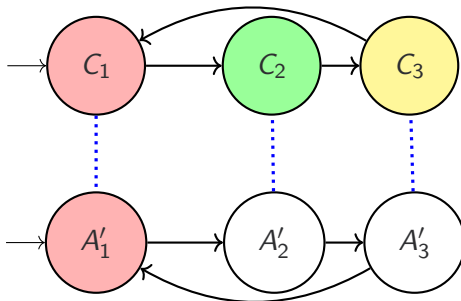
$$S_2 = \delta_C(S_1) \cap \alpha^{-1}(A_2) = \{C_3\} \cap \{C_2, C_3\} = \{C_3\}$$

$$S_3 = \delta_C(S_2) \cap \alpha^{-1}(A_2) = \{C_1\} \cap \{C_2, C_3\} = \emptyset$$

$\alpha^{-1}(A_2) = \{C_2, C_3\}$. We have to split this into those that follow from S_0 ($\{C_2\}$) and those that don't ($\{C_3\}$).

After Splitting

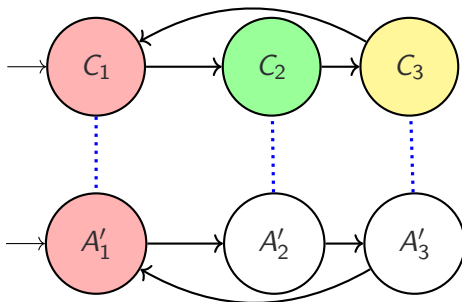
We split A_2 into A'_2 and A'_3



We now have an abstraction that does not exhibit our spurious counterexample, but the state space has increased.

After Splitting

We split A_2 into A'_2 and A'_3

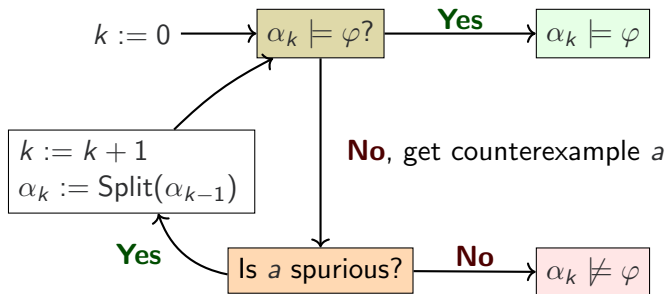


We now have an abstraction that does not exhibit our spurious counterexample, but the state space has increased.

In fact, it's impossible to refine this further, why?

CEGAR

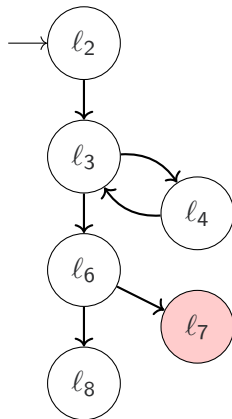
This technique gives us an approach called **Counterexample Guided Abstraction Refinement** (CEGAR). We have a starting abstraction α_0 and an ACTL formula φ :



C Programs

Objective: Prove that our assertion is never violated.

```
1  int main() {  
2      int i = 0, n = 0;  
3      while (i < n) {  
4          i++;  
5      }  
6      if (i < n)  
7          assert(false);  
8  }
```



Need to check reachability, but can we simplify the state space first?

Predicate Abstraction

Predicate Abstraction

A *predicate abstraction* of a program is a version of the program with the same control flow graph, where all variables are replaced with *boolean overapproximations*.

Booleans can be true, false, or * (nondeterministically true or false).

A boolean overapproximation is a predicate that enables *more* behaviours.

Basic PA

To start with, let's try using $i < n$ as our only predicate:

```
1  int main() {  
2      int i = 0, n = 0;  
3      while (i < n) {  
4          i++;  
5      }  
6      if (i < n)  
7          assert(false);  
8  }
```

we want our boolean program to be an **abstraction**.

Requirement

If a location is not reachable in the abstraction, it is not reachable in the concrete program.

Basic PA

To start with, let's try using $i < n$ as our only predicate:

```
1  int main() {  
2      int i = 0, n = 0;  
3      while (i < n) {  
4          i++;  
5      }  
6      if (i < n)  
7          assert(false);  
8  }
```

```
1  int main() {  
2      int b = false;  
3      while (b) {  
4          b = b?*:false;  
5      }  
6      if (b)  
7          assert(false);  
8  }
```

Basic PA

To start with, let's try using $i < n$ as our only predicate:

```
1  int main() {  
2      int i = 0, n = 0;  
3      while (i < n) {  
4          i++;  
5      }  
6      if (i < n)  
7          assert(false);  
8  }
```

```
1  int main() {  
2      int b = false;  
3      while (b) {  
4          b = b?:false;  
5      }  
6      if (b)  
7          assert(false);  
8  }
```

we want our boolean program to be an **abstraction**.

Requirement

If a location is not reachable in the abstraction, it is not reachable in the concrete program.

Harder PA

Now let's try using $i \geq 3$ and $n < 2$ as our only predicates:

```
1  int main() {  
2      int i = 0, n = 0;  
3      while (i < n) {  
4          i++;  
5      }  
6      if (i < n)  
7          assert(false);  
8  }
```

Harder PA

Now let's try using $i \geq 3$ and $n < 2$ as our only predicates:

```
1  int main() {  
2      int i = 0, n = 0;  
3      while (i < n) {  
4          i++;  
5      }  
6      if (i < n)  
7          assert(false);  
8  }
```

```
1  int main() {  
2      int b1 = false, b2 = true;  
3      while (??) {  
4          b1 = b1?*:true;  
5      }  
6      if (??)  
7          assert(false);  
8  }
```

Harder PA

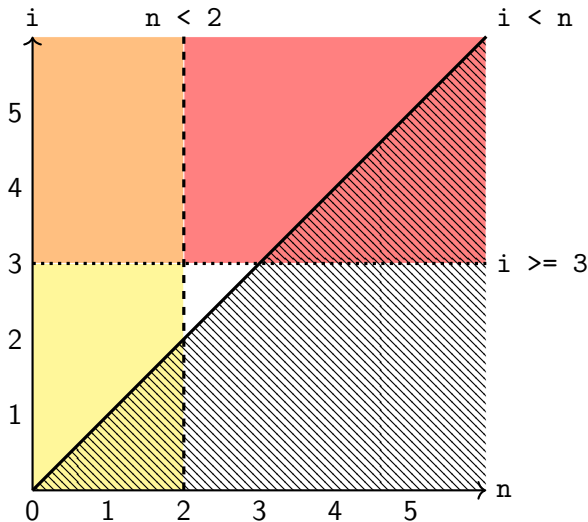
Now let's try using $i \geq 3$ and $n < 2$ as our only predicates:

```
1  int main() {  
2      int i = 0, n = 0;  
3      while (i < n) {  
4          i++;  
5      }  
6      if (i < n)  
7          assert(false);  
8  }
```

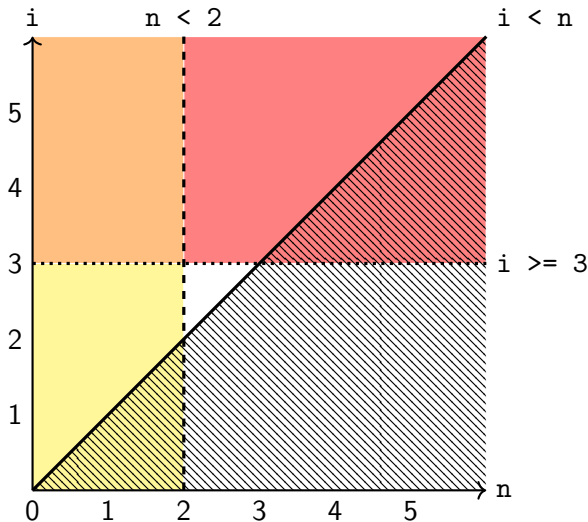
```
1  int main() {  
2      int b1 = false, b2 = true;  
3      while (??) {  
4          b1 = b1?*:true;  
5      }  
6      if (??)  
7          assert(false);  
8  }
```

What do we use for the ?? It must **overapproximate** $i < n$.

Abstract Condition



Abstract Condition



The only overapproximation is $\neg(i \geq 3 \wedge n < 3)$ i.e. $!(b1 \ \&\& \ b2)$

Harder PA

```
1  int main() {  
2      int i = 0, n = 0;  
3      while (i < n) {  
4          i++;  
5      }  
6      if (i < n)  
7          assert(false);  
8  }
```

```
1  int main() {  
2      int b1 = false, b2 = true;  
3      while (!(b1 && b2)){  
4          b1 = b1?:true;  
5      }  
6      if (!(b1 && b2))  
7          assert(false);  
8  }
```

No Predicates

The abstraction with no predicates has **all states reachable**:

```
1  int main() {  
2      int i = 0, n = 0;  
3      while (i < n) {  
4          i++;  
5      }  
6      if (i < n)  
7          assert(false);  
8  }
```

```
1  int main() {  
2      ;;  
3      while (*) {  
4          ;;  
5      }  
6      if (*)  
7          assert(false);  
8  }
```

How do we find out what predicates to add?

No Predicates

The abstraction with no predicates has **all states reachable**:

```
1  int main() {  
2      int i = 0, n = 0;  
3      while (i < n) {  
4          i++;  
5      }  
6      if (i < n)  
7          assert(false);  
8  }
```

```
1  int main() {  
2      ;;  
3      while (*) {  
4          ;;  
5      }  
6      if (*)  
7          assert(false);  
8  }
```

How do we find out what predicates to add? **Use CEGAR!**

No Predicates

The abstraction with no predicates has **all states reachable**:

```
1  int main() {  
2      int i = 0, n = 0;  
3      while (i < n) {  
4          i++;  
5      }  
6      if (i < n)  
7          assert(false);  
8  }
```

```
1  int main() {  
2      ;;  
3      while (*) {  
4          ;;  
5      }  
6      if (*)  
7          assert(false);  
8  }
```

How do we find out what predicates to add? **Use CEGAR!**

Example (Abstract Counterexample)

Lines 3 $\rightarrow$ 6 $\rightarrow$ 7.

No Predicates

The abstraction with no predicates has **all states reachable**:

```
1  int main() {  
2      int i = 0, n = 0;  
3      while (i < n) {  
4          i++;  
5      }  
6      if (i < n)  
7          assert(false);  
8  }
```

```
1  int main() {  
2      ;;  
3      while (*){  
4          ;;  
5      }  
6      if (*)  
7          assert(false);  
8  }
```

How do we find out what predicates to add? **Use CEGAR!**

Example (Abstract Counterexample)

Lines 3 $\rightarrow$ 6 $\rightarrow$ 7. Looking at the concrete program, this path would require $i \geq n$ (to move from line 3 to 6) and $i < n$ (to move from line 6 to 7).

No Predicates

The abstraction with no predicates has **all states reachable**:

```
1  int main() {  
2      int i = 0, n = 0;  
3      while (i < n) {  
4          i++;  
5      }  
6      if (i < n)  
7          assert(false);  
8  }
```

```
1  int main() {  
2      ;;  
3      while (*) {  
4          ;;  
5      }  
6      if (*)  
7          assert(false);  
8  }
```

How do we find out what predicates to add? **Use CEGAR!**

Example (Abstract Counterexample)

Lines 3 $\rightarrow$ 6 $\rightarrow$ 7. Looking at the concrete program, this path would require $i \geq n$ (to move from line 3 to 6) and $i < n$ (to move from line 6 to 7).

Both can't be true simultaneously. This path is **spurious**.

Interpolants

Craig's Interpolation Theorem

If we have two predicates $P(x)$ and $Q(y)$ such are contradictory (i.e. $\neg(P(x) \wedge Q(y))$), then there exists a predicate $I(x \cap y)$ which:

- is implied by $P(x)$, i.e. $P(x) \Rightarrow I(x \cap y)$, and
- contradicts $Q(y)$ i.e. $\neg(I(x \cap y) \wedge Q(y))$.

Crucially, the interpolant $I(x \cap y)$ only ranges over variables common to both predicates.

Interpolants

Craig's Interpolation Theorem

If we have two predicates $P(x)$ and $Q(y)$ such are contradictory (i.e. $\neg(P(x) \wedge Q(y))$), then there exists a predicate $I(x \cap y)$ which:

- is implied by $P(x)$, i.e. $P(x) \Rightarrow I(x \cap y)$, and
- contradicts $Q(y)$ i.e. $\neg(I(x \cap y) \wedge Q(y))$.

Crucially, the interpolant $I(x \cap y)$ only ranges over variables common to both predicates.

Example

- $(i = 1)$ and $i \leq 0$:

Interpolants

Craig's Interpolation Theorem

If we have two predicates $P(x)$ and $Q(y)$ such are contradictory (i.e. $\neg(P(x) \wedge Q(y))$), then there exists a predicate $I(x \cap y)$ which:

- is implied by $P(x)$, i.e. $P(x) \Rightarrow I(x \cap y)$, and
- contradicts $Q(y)$ i.e. $\neg(I(x \cap y) \wedge Q(y))$.

Crucially, the interpolant $I(x \cap y)$ only ranges over variables common to both predicates.

Example

- $(i = 1)$ and $i \leq 0$: $i > 0$
- $(i \leq 2 \wedge k = i + 1)$ and $k > 5$:

Interpolants

Craig's Interpolation Theorem

If we have two predicates $P(x)$ and $Q(y)$ such are contradictory (i.e. $\neg(P(x) \wedge Q(y))$), then there exists a predicate $I(x \cap y)$ which:

- is implied by $P(x)$, i.e. $P(x) \Rightarrow I(x \cap y)$, and
- contradicts $Q(y)$ i.e. $\neg(I(x \cap y) \wedge Q(y))$.

Crucially, the interpolant $I(x \cap y)$ only ranges over variables common to both predicates.

Example

- $(i = 1)$ and $i \leq 0$: $i > 0$
- $(i \leq 2 \wedge k = i + 1)$ and $k > 5$: $k \leq 4$
- $(i \geq n)$ and $i < n$:

Interpolants

Craig's Interpolation Theorem

If we have two predicates $P(x)$ and $Q(y)$ such are contradictory (i.e. $\neg(P(x) \wedge Q(y))$), then there exists a predicate $I(x \cap y)$ which:

- is implied by $P(x)$, i.e. $P(x) \Rightarrow I(x \cap y)$, and
- contradicts $Q(y)$ i.e. $\neg(I(x \cap y) \wedge Q(y))$.

Crucially, the interpolant $I(x \cap y)$ only ranges over variables common to both predicates.

Example

- $(i = 1)$ and $i \leq 0$: $i > 0$
- $(i \leq 2 \wedge k = i + 1)$ and $k > 5$: $k \leq 4$
- $(i \geq n)$ and $i < n$: $i \geq n$

Path Interpolant

Sequence of program locations

$$\ell_1 \ell_2 \ell_3 \dots \ell_k$$

Path Interpolant

Sequence of program locations

$$\ell_1 \ell_2 \ell_3 \dots \ell_k$$


Sequence of predicates

$$\pi_1 \pi_2 \pi_3 \dots \pi_k$$

Path Interpolant

Sequence of program locations

$$l_1 l_2 l_3 \dots l_k$$


Sequence of predicates

$$\pi_1 \pi_2 \pi_3 \dots \pi_k$$

$$\pi_1 \wedge \pi_2 \wedge \pi_3 \dots \pi_k$$

Path Interpolant

Sequence of program locations

$l_1 l_2 l_3 \dots l_k$



Sequence of predicates

$\pi_1 \pi_2 \pi_3 \dots \pi_k$



$\pi_1 \wedge \pi_2 \wedge \pi_3 \dots \pi_k$

SAT



Non-spurious case

Path Interpolant

Sequence of program locations

$l_1 l_2 l_3 \dots l_k$



Sequence of predicates

$\pi_1 \pi_2 \pi_3 \dots \pi_k$



$\pi_1 \wedge \pi_2 \wedge \pi_3 \dots \pi_k$

SAT

UNSAT

Non-spurious case

Spurious case

Path Interpolant

Sequence of program locations

$l_1 l_2 l_3 \dots l_k$



Sequence of predicates

$\pi_1 \pi_2 \pi_3 \dots \pi_k$



$\pi_1 \wedge \pi_2 \wedge \pi_3 \dots \pi_k$

SAT

UNSAT

Non-spurious case

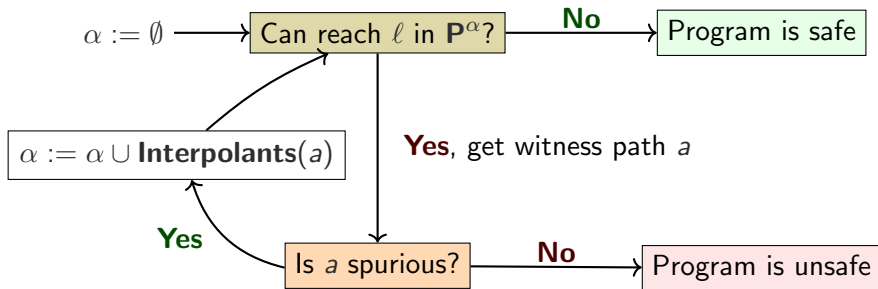
Spurious case

There exists interpolants

$l_1 l_2 l_3 \dots l_{k-1}$

CEGAR for C Programs

Let $\mathbf{P}$ be our program, α be our predicate set, and $\mathbf{P}^\alpha$ be the predicate abstraction of $\mathbf{P}$ using α . The location $\ell \in \mathbf{P}$ is our **bad state** we want to avoid (assertion failure).



Termination

On finite automata

- Finite number of states

Termination

On finite automata

- Finite number of states
- Each CEGAR loop increases the number of states in the abstraction, but the number **can't exceed** the number of concrete states.

Termination

On finite automata

- Finite number of states
- Each CEGAR loop increases the number of states in the abstraction, but the number **can't exceed** the number of concrete states.

On C programs

Termination

On finite automata

- Finite number of states
- Each CEGAR loop increases the number of states in the abstraction, but the number **can't exceed** the number of concrete states.

On C programs

- (Effectively) **infinite amount of states**

Termination

On finite automata

- Finite number of states
- Each CEGAR loop increases the number of states in the abstraction, but the number **can't exceed** the number of concrete states.

On C programs

- (Effectively) **infinite amount of states**
- $\therefore$ **No guarantee** of termination
- When it terminates it is both **sound** (in that it always finds errors if they exist) and **complete** (it will not provide spurious errors).

Bibliography

- Baier/Katoen, Sections 7.1 (parts), 7.2 (parts), 7.4, 7.5, 7.6, 7.7
- Rob van Glabbeek, The Linear-Time Branching-Time Spectrum I, Handbook of Process Algebra p. 3-99, Elsevier.
- Rob van Glabbeek, COMP6752 course notes.
- E. Clarke, O. Grumberg, S. Jha, Y. Lu, and H. Veith. Counterexample-guided Abstraction Refinement. In Computer Aided Verification, pages 154-169, 2000
- Thomas A. Henzinger, Ranjit Jhala, Rupak Majumdar and Gregoire Sutre, Software Verification with BLAST. In SPIN Workshop 2003, LNCS 2648, pages 235-239.

Weekly Feedback

I would appreciate any comments/suggestions/requests you have on this week's lectures (and tutorials).



<https://forms.office.com/r/sCi2xjavrS>